

Fast PUF-Based Four-Phase Authentication for UAV Swarms: Protocol Design, OMNeT++ Evaluation, and Hardware Projection with Dual Hash Modes

Md Ziyad Hussain, *Student Member, IEEE*, and Rakesh Matam, *Member, IEEE*

Abstract—Unmanned Aerial Vehicle (UAV) swarms require fast, lightweight, and capture-resilient authentication for real-time coordination. Existing options—RSA/ECC public-key infrastructure (PKI) and pre-shared keys (PSK)—either add millisecond-scale handshake overhead or rely on stored private keys that are exposed when a UAV is physically captured. We present a four-phase Physical Unclonable Function (PUF)-based authentication protocol covering: (i) secure enrollment with a BCH(255,131,18) fuzzy extractor; (ii) UAV–Ground Station (GS) mutual authentication via challenge-response with helper-data-protected PUF output; (iii) full-mesh UAV–UAV peer authentication without GS involvement, using a swarm-wide credential issued in Phase 2 as the root of trust; and (iv) symmetric session-key establishment with optional Curve25519 forward secrecy. The protocol uses only hash, XOR, and PUF primitives—no public-key cryptography—and is fully implemented in OMNeT++ 6.3 for a ten-UAV stadium-surveillance scenario. We characterize two hash primitives back-to-back: SHA3-256 truncated to 160 bits (via OpenSSL EVP) representing software-optimized hashing, and a SPONGENT-160-style lightweight sponge construction representing hardware-targeted lightweight cryptography. Measured Phase 2 latency is 0.699 ms (SHA3) and 14.044 ms (SPONGENT); Phase 3 per-pair latency is 0.476 ms and 14.535 ms; communication overhead is 175 B and 104 B respectively. All ten UAV–GS authentications and all 45 peer pairs succeed in both modes. Per-message timing breakdowns localize cost to PUF evaluation, BCH decoding, and hash operations, and a synthesis-based hardware projection shows the SHA3–SPONGENT gap collapsing from 20–30 $\times$ in software to 2–5 $\times$ on FPGA/ASIC. We give five security theorems covering mutual authentication, replay resistance, peer authentication, MITM resistance, and physical capture. Our SHA3 mode is 10–21 $\times$ faster than RSA/ECC baselines while eliminating the stored-key attack surface entirely.

Index Terms—UAV swarm authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, lightweight cryptography, fuzzy extractor, BCH error correction, OMNeT++, peer-to-peer authentication, session key, hardware projection.

I. INTRODUCTION

UNMANNED Aerial Vehicles (UAVs) are reshaping defense, public safety, logistics, agriculture, and disaster-response operations [1]. Multi-UAV swarms have become the standard operational unit: ten or more drones cooperating to surveil a stadium, patrol a border, deliver supplies, or

coordinate a search-and-rescue mission. Such swarms communicate continuously with a Ground Station (GS) over a vertical command-and-control link and with each other over horizontal peer links to maintain formation, share sensor fusion, and route around link outages.

A fundamental prerequisite for any of this is *mutual authentication*: every UAV must verify the identity of its GS and of each peer before accepting commands, exchanging telemetry, or relaying messages. Without it, an adversary can inject a rogue UAV into the swarm, intercept sensitive data, issue unauthorized flight commands, or perform man-in-the-middle attacks on inter-UAV channels.

Conventional authentication options are poorly suited to this regime. **RSA-2048 PKI** requires 8–15 ms per signature verification on an ARM Cortex-A72 [2]; for full-mesh authentication of a 10-UAV swarm, the cumulative cost climbs to nearly a second, and certificates add 256–512 B per exchange. **ECC-256** [3] halves the latency but still requires private keys held in non-volatile memory—recoverable from a captured UAV by side-channel or cold-boot attacks. **Pre-shared keys (PSK)** reach sub-millisecond latency but compromise the entire swarm if any device is captured and require $O(N^2)$ pairwise keys for a swarm of size N . **Blockchain-based** approaches incur seconds of consensus latency [22], incompatible with real-time UAV coordination.

Physical Unclonable Functions (PUFs) are a fundamentally different cryptographic primitive. A PUF maps challenges to responses using the manufacturing-process variations intrinsic to a specific integrated circuit—variations at the nanometer scale [6]. PUFs are *unclonable* (cannot be replicated even by the manufacturer), *unpredictable* (responses to unseen challenges are cryptographically unrelated to prior responses), *tamper-evident* (invasive probing destroys the response), and *lightweight* (an Arbiter PUF needs $\sim 1,200$ gate-equivalents). Crucially, no key is stored: the “key” is the silicon itself, eliminating the dominant attack surface of PKI and PSK schemes.

The challenge in deploying PUFs is that responses are *noisy*: environmental drift, supply-voltage variation, and aging produce bit-flip rates of 3–10% between successive evaluations of the same challenge. Fuzzy extractors [15], [16], typically built on BCH error-correcting codes, recover the exact enrollment response from the noisy field response without leaking the

The authors are with the Department of Computer Science and Engineering, Indian Institute of Information Technology Guwahati, Bongora, Assam 781015, India (e-mail: ziyad.hussain23b@iiitg.ac.in; rakesh@iiitg.ac.in).

Manuscript prepared April 2026.

Contributions

This paper makes the following contributions: leftmargin=*

- 1) **Four-phase protocol design.** We present and formalize a PUF-based authentication framework covering enrollment, UAV-GS mutual authentication, full-mesh UAV-UAV peer authentication, and session-key derivation. Phase 3 avoids GS round-trips entirely by using a swarm-wide network credential, issued during Phase 2, as the shared root of trust.
- 2) **Reference implementation.** We implement the protocol in OMNeT++ 6.3 with a composite link-delay model (propagation, serialization, processing, jitter), a 128-stage Arbiter PUF simulator, a from-scratch BCH(255,131,18) fuzzy extractor with Berlekamp-Massey/Chien decoding, and dual-mode hash benchmarking infrastructure.
- 3) **Dual hash characterization.** We characterize the protocol back-to-back with SHA3-256 (truncated to 160 bits) and a SPONGENT-160-style lightweight sponge, exposing the software/hardware crossover point empirically.
- 4) **End-to-end evaluation.** On a 10-UAV stadium-surveillance scenario with 500×300 m coverage, we measure Phase 2 (0.699/14.044 ms SHA3/SPONGENT) and Phase 3 (0.476/ 14.535 ms per pair) at 100% success across 10 UAV-GS and 45 peer authentications, with per-message latency decomposition.
- 5) **Security analysis.** We give five theorems covering Phase 2 mutual authentication, replay resistance, Phase 3 peer authentication, MITM resistance, and physical capture resistance.
- 6) **Hardware projection.** Substituting published FPGA/ASIC hash synthesis numbers into the per-message timing breakdown, we show the SHA3-SPONGENT gap collapses from 20–30× in software to 2–5× in hardware, making SPONGENT a clear winner for ASIC-integrated UAV security at ~8,390 GE total.

Paper Organization

Section II reviews PUFs, fuzzy extractors, and lightweight hash functions. Section III surveys related work. Section IV presents our four-phase protocol. Section V details the OMNeT++ reference implementation. Section VI reports measured results. Section VII compares to baselines. Section VIII projects performance onto FPGA/ASIC platforms. Section IX provides formal security analysis. Section X discusses limitations and deployment considerations. Section XI concludes.

II. BACKGROUND

A. Physical Unclonable Functions

A PUF is a hardware primitive $\mathcal{P} : \{0,1\}^m \rightarrow \{0,1\}^n$ that maps an m -bit challenge C to an n -bit response R using the intrinsic, sub-nanometer manufacturing variations of a specific integrated circuit. These variations arise from uncontrollable processes during fabrication—transistor threshold-voltage spread, wire-width fluctuation,

oxide-thickness mismatch—and are *unique*, *random*, *permanent*, and *unclonable* [6]. A pair (C, R) with $R = \mathcal{P}(C)$ is a challenge-response pair (CRP).

Arbiter PUF. An m -stage Arbiter PUF [7] consists of m delay stages, each with two paths of marginally different propagation delay. A challenge bit selects path swapping at each stage; a final arbiter latches whichever path's edge arrives first. The output bit is a sign function of the accumulated delay difference. A 128-stage Arbiter PUF costs ~1,200 GE and evaluates in ~30 ns.

Ring Oscillator PUF. An RO PUF uses N nominally identical ring oscillators whose frequencies differ due to manufacturing variation [8], [9]. More environmentally stable than Arbiter PUFs but consumes 3,000–5,000 GE.

SRAM PUF. Exploits power-up states of SRAM cells determined by transistor mismatch [10]. Reuses existing memory but restricted to power-up and limited CRP space.

PUF metrics. Quality is characterized by *uniqueness* (inter-chip Hamming distance, ideally 50%), *reliability* (intra-chip Hamming distance, ideally 0% but in practice 3–10%), and *unpredictability* (responses to unseen challenges are statistically independent of prior CRPs).

B. Fuzzy Extractors and BCH Error Correction

A fuzzy extractor extracts a stable key from a noisy source. The *Gen* algorithm produces helper data H during enrollment; the *Rep* algorithm recovers the enrollment value from a noisy re-evaluation using H . For PUFs, the standard construction uses a linear error-correcting code: enrollment computes

$$H = \text{Encode}(R) \oplus R \quad (1)$$

and field recovery computes

$$\text{ecc} = H \oplus R^{\text{noisy}}, \quad (2)$$

$$R = \text{Decode}(R^{\text{noisy}}, \text{ecc}). \quad (3)$$

Under the assumption that Encode is a linear code over $\mathbb{F}_2$ and that R has sufficient min-entropy, this construction is known to preserve secrecy of R given H [16].

We use **BCH(255, 131, 18)**: a 255-bit codeword carrying 131 information bits and correcting up to $t=18$ errors over $\text{GF}(2^8)$. For our 128-bit responses at 3% BER (Bernoulli(0.03) per bit), expected errors per evaluation are $128 \times 0.03 = 3.84$, giving a ~4.7× margin to the BCH correction capacity. The probability that the bit-error count exceeds 18 in a 128-bit block under this model is $< 10^{-15}$ by Chernoff bound, so decoding-failure events are effectively impossible in steady state.

BCH decoding consists of syndrome computation, the Berlekamp-Massey algorithm to obtain the error-locator polynomial, and Chien search to extract the error positions; the BCH(255,131,18) decoder costs ~5,000 GE in hardware with ~2 μs latency [10].

C. Lightweight Hash Functions

Two hash designs anchor our evaluation:

SHA3-256. The NIST-standardized Keccak permutation hash [12], [13] optimized for 64-bit software execution.

Hardware-accelerated implementations on modern x86 reach $\sim 0.4 \mu\text{s}$ per call. Hardware (ASIC) area is $\sim 38,000$ GE—substantially larger than lightweight alternatives. Used in this paper truncated to 160 bits.

SPONGENT-160. Introduced by Bogdanov *et al.* [11], SPONGENT is a sponge-based hash with a PRESENT-type 4-bit S-box, a bit-permutation that requires no logic gates (only wire routing) in hardware, and parameters $r=16$, $c=144$, 80 rounds for the 160-bit variant. ASIC area is $\sim 2,190$ GE—one of the smallest known hash implementations [14]. Per-call hardware latency on a 100 MHz FPGA is 5–10 μs ; software cost is high because the bit-level permutation maps poorly to byte-wise CPU instructions.

The asymmetry between these two designs is precisely the phenomenon our dual-mode evaluation isolates.

III. RELATED WORK

A. PUF-Based UAV Authentication

Gope and Sikdar [2] proposed a hash/XOR-only PUF authentication for smart UAV networks reporting 10–50 ms latency on a Raspberry Pi 4B, but addressing only UAV–GS authentication without peer-to-peer capability. Li *et al.* [4] extended PUF authentication to cross-domain emergency-rescue UAVs at the cost of 640 B overhead and 45–80 ms latency on embedded ARM. Khan *et al.* [5] combined PUF with biometrics for 6G-enabled UAV networks; their formal real-or-random (ROR) model security analysis provides a strong verification framework. Dey and Hossain [19] proposed MITM- and DoS-resistant PUF authentication via sensor-initiated registration. Hassan *et al.* [20] catalog common pitfalls in PUF protocol designs (insufficient nonce management, helper-data leakage). Xu *et al.* [7] introduce dynamic CRP rotation to prevent database exhaustion, which we adopt.

None of these works characterizes a full-mesh PUF-based UAV peer-authentication scheme end-to-end in a discrete-event network simulator with per-message timing decomposition.

B. Peer-to-Peer UAV Authentication

Direct UAV–UAV authentication remains under-explored. Bera *et al.* [3] achieve sub-millisecond P2P authentication using pre-shared symmetric keys, but lack the PUF hardware root of trust and rely on key storage. Banerjee *et al.* [21] identify vulnerabilities in prior UAV–GS authentication schemes (impersonation, session-key compromise, lack of forward secrecy), motivating our explicit security theorems and the optional ECDH extension for forward secrecy.

C. Lightweight Cryptography

Eisenbarth *et al.* [14] comprehensively evaluate lightweight hash, block-cipher, and MAC primitives across software and hardware, establishing the 2,190-GE figure for SPONGENT-160 used in our hardware projection. SHA3/Keccak [12] is software-fast but requires $\sim 38,000$ GE in hardware— $17\times$ larger than SPONGENT.

TABLE I
COMPARISON OF REPRESENTATIVE UAV/IoT AUTHENTICATION APPROACHES.

Approach	PUF	P2P	Light	FwdSec	ErrCor	Latency
RSA-2048 PKI	✗	✓	✗	opt	N/A	8–15 ms
ECC-256	✗	✓	mod.	opt	N/A	4–8 ms
PSK (AES-GCM)	✗	✓	✓	✗	N/A	0.3–0.5 ms
Gope [2]	✓	✗	✓	✗	simple	10–50 ms
Chatterjee [4]	✓	part.	mod.	✗	BCH	45–80 ms
Khan [5]	✓	✗	mod.	✓	BCH	5–20 ms
Bera [3]	✗	✓	✓	✗	N/A	< 1 ms
This work	✓	✓	✓	opt	BCH	0.67 ms

D. Comparison Summary

Table I summarizes the comparison.

Our protocol is the only entry combining PUF hardware root of trust, direct peer-to-peer authentication, hash-and-XOR-only computation, optional forward secrecy, robust BCH error correction, and sub-millisecond latency.

IV. PROPOSED FOUR-PHASE PROTOCOL

A. System and Threat Model

The system consists of one trusted Ground Station GS and N UAVs $\{\text{UAV}_i\}_{i=1}^N$. The GS has substantial computational resources and tamper-resistant storage. Each UAV is equipped with a 128-stage Arbiter PUF, a lightweight processor, a wireless transceiver, and minimal non-volatile memory.

We adopt the Dolev-Yao adversarial model [17] augmented with physical capture: leftmargin=*

- **Network adversary:** can eavesdrop, intercept, modify, replay, delay, or drop arbitrary messages on the wireless channel.
- **Physical capture:** may capture a UAV and extract NVM contents but cannot clone PUF hardware—invasive probing destroys the PUF [18].
- **Computational bounds:** polynomially-bounded; cannot invert hashes or break PUF unclonability/unpredictability except with negligible probability.
- **Side channels:** standard countermeasures assumed (constant-time hashing, masking against power analysis).

B. Notation

Table II summarizes the symbols used throughout.

C. Phase 1: Secure Enrollment

Phase 1 is performed once per UAV in a physically secure facility (wired channel, shielded RF cage). For each UAV_i , the GS performs:

1.1 CRP generation. Generate $n \geq 12$ random 128-bit challenges $\{C_i^{(k)}\}_{k=1}^n$ using a cryptographic RNG. For each, evaluate the UAV's PUF in situ over the secure channel: $R_i^{(k)} = \text{PUF}_i(C_i^{(k)})$.

1.2 Helper data. Compute BCH helper data per Eq. (1):

$$H_i^{(k)} = \text{BCH_encode}(R_i^{(k)}) \oplus R_i^{(k)}. \quad (4)$$

Under Maes' helper-data-masking analysis [16], H is computationally independent of R : an adversary observing H gains no information about the PUF response.

TABLE II
PROTOCOL NOTATION.

Symbol	Description
UAV_i	The i -th UAV in the swarm
GS	Ground Station
TID_i	Temporary identity of UAV_i
$C_i^{(k)}$	k -th 128-bit PUF challenge for UAV_i
$R_i^{(k)} = \text{PUF}_i(C_i^{(k)})$	Noisy 128-bit PUF response
$H_i^{(k)}$	BCH helper data for $C_i^{(k)}$
$h(\cdot)$	160-bit hash (SHA3 or SPONGENT)
$\oplus$	Bitwise XOR
$\parallel$	Concatenation
T_x	32-bit timestamp
N_x, n_x	128-bit random nonces
SK_i	Phase-2 UAV-GS session key
SK_{ij}	Phase-3 peer session key
Cred	20-byte swarm-wide network credential

Algorithm 1 Phase 2 UAV-GS Mutual Authentication

- 1: **UAV** $\rightarrow$ **GS**: $M_1 = \langle TID_i, T_1, N_1, h(TID_i \parallel T_1 \parallel N_1) \rangle$
- 2: **GS**: verify h , look up TID_i , fetch $CRP(C_i^{(k)}, H_i^{(k)}, R_i^{(k)})$, draw N_2
- 3: **GS** $\rightarrow$ **UAV**: $M_2 = \langle C_i^{(k)}, N_2, h(TID_i \parallel C_i^{(k)} \parallel N_2 \parallel T_1) \rangle$
- 4: **UAV**: verify GS hash, evaluate $R^{\text{noisy}} \leftarrow \text{PUF}_i(C_i^{(k)})$
- 5: **UAV**: $R^{\text{masked}} \leftarrow R^{\text{noisy}} \oplus h(N_2)$;
 $SK_i \leftarrow h(R^{\text{noisy}} \parallel N_1 \parallel N_2 \parallel T_1)$
- 6: **UAV** $\rightarrow$ **GS**: $M_3 = \langle R^{\text{masked}}, h(R^{\text{masked}} \parallel SK_i) \rangle$
- 7: **GS**: recover R^{noisy} , compute $\text{ecc} = H_i^{(k)} \oplus R^{\text{noisy}}$, decode $R \leftarrow \text{BCH_decode}(R^{\text{noisy}}, \text{ecc})$
- 8: **GS**: check $\text{HD}(R, R_i^{(k)}) \leq \tau$, recompute SK_i , verify M_3 hash
- 9: **GS** $\rightarrow$ **UAV**: $M_4 = \langle \text{success}, \text{Cred} \rangle$ (if successful)

1.3 Identity assignment. Assign a temporary identity TID_i (e.g., a 64-bit string) used in subsequent wireless exchanges, providing pseudonymity against passive observers.

1.4 Storage. The GS stores $\{TID_i, \{(C_i^{(k)}, H_i^{(k)}, R_i^{(k)})\}_{k=1}^n\}$ in a tamper-resistant database. The UAV stores only TID_i and its own PUF_i instance—no key material whatsoever.

D. Phase 2: UAV-GS Mutual Authentication

Phase 2 is a four-message protocol executed when a UAV joins the operational swarm. Algorithm 1 summarizes the exchange; the detailed equations follow.

Message M_1 (UAV $\rightarrow$ GS). The UAV draws a fresh nonce and sends an authenticated request:

$$N_1 \xleftarrow{\$} \{0, 1\}^{128} \quad (5)$$

$$\alpha_1 = h(TID_i \parallel T_1 \parallel N_1) \quad (6)$$

$$M_1 = \langle TID_i, T_1, N_1, \alpha_1 \rangle. \quad (7)$$

Message M_2 (GS $\rightarrow$ UAV). The GS verifies α_1 , rejects stale timestamps ($|T_{\text{now}} - T_1| > \Delta T_{\text{max}}$, default 5 s), looks up

Algorithm 2 Phase 3 UAV-UAV Peer Authentication

- 1: **UAV $_i$** : $n_i \xleftarrow{\$} \{0, 1\}^{128}$; $\tau_1 = h(i \parallel j \parallel n_i \parallel \text{Cred})$
- 2: **UAV $_i$** $\rightarrow$ **UAV $_j$** : $P_1 = \langle n_i, \tau_1 \rangle$
- 3: **UAV $_j$** : verify τ_1 ; $n_j \xleftarrow{\$} \{0, 1\}^{128}$; $\tau_2 = h(j \parallel i \parallel n_i \parallel n_j \parallel \text{Cred})$
- 4: **UAV $_j$** $\rightarrow$ **UAV $_i$** : $P_2 = \langle n_j, \tau_2 \rangle$
- 5: **UAV $_i$** : verify τ_2 ; $\tau_3 = h(i \parallel j \parallel n_j \parallel \text{Cred})$; $SK_{ij} = h(i \parallel j \parallel n_i \parallel n_j \parallel \text{Cred})$
- 6: **UAV $_i$** $\rightarrow$ **UAV $_j$** : $P_3 = \langle \tau_3 \rangle$
- 7: **UAV $_j$** : verify τ_3 ; derive SK_{ij}

TID_i , fetches an unused CRP, and replies:

$$N_2 \xleftarrow{\$} \{0, 1\}^{128} \quad (8)$$

$$\beta_1 = h(TID_i \parallel C_i^{(k)} \parallel N_2 \parallel T_1) \quad (9)$$

$$M_2 = \langle C_i^{(k)}, N_2, \beta_1 \rangle. \quad (10)$$

Message M_3 (UAV $\rightarrow$ GS). The UAV verifies β_1 , evaluates its PUF, derives the session key, masks the response, and constructs an authentication token:

$$R^{\text{noisy}} = \text{PUF}_i(C_i^{(k)}) \quad (11)$$

$$R^{\text{masked}} = R^{\text{noisy}} \oplus h(N_2) \quad (12)$$

$$SK_i = h(R^{\text{noisy}} \parallel N_1 \parallel N_2 \parallel T_1) \quad (13)$$

$$\gamma_1 = h(R^{\text{masked}} \parallel SK_i) \quad (14)$$

$$M_3 = \langle R^{\text{masked}}, \gamma_1 \rangle. \quad (15)$$

Message M_4 (GS $\rightarrow$ UAV). The GS recovers $R^{\text{noisy}} = R^{\text{masked}} \oplus h(N_2)$, performs BCH correction using the fuzzy-extractor Rep procedure:

$$\text{ecc} = H_i^{(k)} \oplus R^{\text{noisy}} \quad (16)$$

$$R = \text{BCH_decode}(R^{\text{noisy}}, \text{ecc}), \quad (17)$$

checks the Hamming distance against the enrolled response ($\text{HD}(R, R_i^{(k)}) \leq \tau$, default $\tau=24$ bits), recomputes SK_i , verifies γ_1 , and on success issues a swarm-wide 160-bit network credential Cred to the UAV. The GS then advances to the next CRP for UAV_i to prevent CRP reuse.

E. Phase 3: UAV-UAV Peer Authentication

After successful Phase 2, each UAV holds the shared swarm credential Cred . Phase 3 performs full-mesh peer authentication: for each ordered pair (i, j) with $i < j$, the two UAVs run a three-message exchange (Alg. 2).

The mathematics:

$$\tau_1 = h(i \parallel j \parallel n_i \parallel \text{Cred}) \quad (18)$$

$$\tau_2 = h(j \parallel i \parallel n_i \parallel n_j \parallel \text{Cred}) \quad (19)$$

$$\tau_3 = h(i \parallel j \parallel n_j \parallel \text{Cred}) \quad (20)$$

$$SK_{ij} = h(i \parallel j \parallel n_i \parallel n_j \parallel \text{Cred}). \quad (21)$$

Possession of Cred is gated by Phase 2 success—hence by PUF possession—so Cred acts as a PUF-derived shared secret that any swarm-resident UAV can use to authenticate to any other. The token chain (τ_1, τ_2, τ_3) binds both nonces and

both identities, providing mutual authentication; the symmetric form of SK_{ij} ensures both endpoints compute the same session key.

F. Phase 4: Session Key and Optional Forward Secrecy

Phase 4 is implicit: SK_i from Phase 2 and SK_{ij} from Phase 3 are the established session keys. Their 160-bit width is suitable as HKDF input for 128-bit AES-GCM session encryption.

Optional ECDH forward secrecy. For applications requiring perfect forward secrecy (PFS), Phase 4 can incorporate an ephemeral Curve25519 exchange [23]:

$$SK_{ij}^{\text{PFS}} = h(SK_{ij} \| g^{a_i \cdot b_j}), \quad (22)$$

where a_i, b_j are ephemeral ECDH private keys and $g^{a_i b_j}$ is the shared ECDH secret. This adds ~ 2 ms of compute on ARM Cortex-A72 but ensures that compromising Cred at time t does not compromise session keys established before t . The reference implementation gates this feature on a compile-time `USE_SODIUM` flag.

V. OMNET++ REFERENCE IMPLEMENTATION

A. Discrete-Event Network Simulator

We implemented the protocol in OMNeT++ 6.3 [24], a discrete-event C++ network-simulation framework widely used for protocol research. The simulation comprises three OMNeT++ simple modules: `leftmargin=*`

- `UAVNode`: runs Phase 2 as client and Phase 3 as both initiator and responder.
- `GroundStation`: runs Phase 1 in `initialize()`, then Phase 2 verification, BCH decoding, and credential issuance.
- `NetworkController`: orchestrates simulation lifecycle.

Cryptographic primitives sit in a `src/crypto/` library; the protocol logic is in `src/protocols/`. All modules share a single `HashWrapper` that selects between SHA3 and SPONGENT at runtime and maintains per-mode timing accumulators.

B. Link-Delay Model

Inter-module messaging uses OMNeT++'s `sendDirect()` with a composite delay computed at send time:

$$d = \underbrace{\frac{\text{dist}}{c}}_{d_{\text{prop}}} + \underbrace{\frac{8L}{r}}_{d_{\text{ser}}} + d_{\text{proc}} + d_{\text{jitter}}, \quad (23)$$

where L is payload bytes, $r = 6$ Mbps (802.11a base-line), $d_{\text{proc}} = 0.1$ ms (mean processing delay), and $d_{\text{jitter}} \sim |\mathcal{N}(0, 0.02^2)|$ ms half-Gaussian queue jitter. Inter-node distances are read from each module's display-coordinate tag. MAC-layer contention (CSMA/CA, RTS/CTS, ACKs, retransmissions) is *not* modeled; the reported network delay is therefore a lower bound. As Section X discusses, this does not affect the relative comparisons that are the focus of this work.

C. Cryptographic Primitives

PUF. A 128-stage Arbiter-style simulator. Each challenge is hashed (FNV-1a) into a device-seeded `mt19937` state, and 128 response bits are generated; a Bernoulli(p) noise channel flips bits uniformly with $p = 0.03$. Empirically, the inter-chip Hamming distance across simulated devices is 45–55% and intra-chip is $\leq 5\%$, consistent with the Arbiter PUF literature [7].

BCH(255,131,18). From-scratch $GF(2^8)$ codec with primitive polynomial $x^8 + x^4 + x^3 + x^2 + 1$. Encoding generates 144-bit parity from 128-bit data. Decoding follows the syndrome–Berlekamp–Massey–Chien-search pipeline standard for BCH [10], correcting up to $t=18$ errors. The implementation is self-contained (no `libcorrect` dependency) but can be substituted at build time via the `USE_LIBCORRECT` macro.

SHA3-256. Via OpenSSL's EVP API (`EVP_sha3_256`), truncated to 160 bits to match the SPONGENT output width and the protocol-level $h(\cdot)$ specification. Steady-state per-call cost on the test platform is ~ 0.4 μ s.

SPONGENT-160. A lightweight sponge construction with the PRESENT-type 4-bit S-box (Table 1 of [11]), $r=16$, $c=144$, 80 rounds, and a compact bit-permutation suitable for relative timing characterization¹. Per-call software cost is ~ 890 μ s.

Hash wrapper warmup. OpenSSL's first `EVP_sha3_256` invocation triggers lazy loading via `EVP_MD_fetch()`— ~ 2.8 ms on first call. Both `UAVNode::initialize()` and `GroundStation::initialize()` issue one throwaway hash and then reset all timing counters, ensuring steady-state measurement.

D. Per-Message Timing Instrumentation

Every Phase 2/Phase 3 step is bracketed by `std::chrono::steady_clock` start/stop pairs. The hash wrapper maintains `sha3TimeMs` and `spongencTimeMs` accumulators incremented on every call. Each protocol message carries `metricA–metricD` fields so the receiver can record both endpoints' timings when finalizing the exchange. The result is a per-UAV scalar dump containing ~ 30 fine-grained timing metrics per Phase 2 exchange and ~ 10 metrics per Phase 3 pair.

E. Stadium-Surveillance Scenario

We model a representative ten-UAV stadium-perimeter deployment: 500×300 m playing field; ten UAVs distributed around the perimeter at altitude ~ 50 m (the exact positions are encoded as display-coordinate tags in the NED file); GS at the geometric center (275, 175) m. UAV–GS distances range from ~ 175 m to ~ 325 m; UAV–UAV distances range from

¹The full SPONGENT-160 bit-permutation is trivial in hardware (deterministic wire routing) but software-unfriendly. Our implementation preserves the round count, S-box, rate, capacity, and order-of-magnitude software cost; a full-spec evaluation against a verified reference implementation is deferred to future work.

TABLE III
PHASE 2 UAV-GS AUTHENTICATION: AGGREGATE RESULTS ($N=10$ UAVs).

Metric	SHA3-256	SPONGENT-160
UAV-side compute (μ s)	7.2	6,703
of which hash (μ s)	3.4	6,697
GS-side compute (μ s)	26.9	6,676
Network delay (μ s)	664.6	664.6
End-to-end latency (ms)	0.699	14.044
Communication overhead (B)	175	175
Authentication success rate	10/10	10/10

~ 94 m to ~ 583 m, yielding propagation delays of $0.3\text{--}2\ \mu\text{s}$ —two orders of magnitude smaller than the modeled processing delay.

Phase 2 start times are staggered by 0.5 s between UAVs to avoid artificial bursts; Phase 3 full-mesh authentication is launched 10 s after the corresponding Phase 2 succeeds. The simulation runs for 120 s.

F. Build and Run

Build is a thin wrapper around `opp_makemake`, producing the binary `src/uavauthsim`. Two named configurations exercise the dual hash modes: `StadiumSHA3` and `StadiumSPONGENT`. Each writes scalar results to `simulations/results/`; a Python exporter parses these into per-phase CSV files for analysis.

VI. EVALUATION

All measurements are from the `StadiumSHA3` and `StadiumSPONGENT` configurations, `release-build (-O3)`, on an x86-64 laptop. Reported values are means over the simulation population (10 UAVs for Phase 2, 45 peer pairs for Phase 3). Detailed per-UAV and per-pair CSVs are available in the project repository.

A. Phase 2 Aggregate Results

Table III reports Phase 2 metrics with full component decomposition.

Observations. Total compute is broken into UAV-side and GS-side contributions because both add to wall-clock latency in this single-CPU-per-node model. In SHA3 mode, compute (UAV + GS = $34.1\ \mu\text{s}$) is 4.9% of latency; network delay dominates at 95% . In SPONGENT mode, compute ($13,379\ \mu\text{s}$) is 95% of latency. Network delay is identical between modes because message sizes are identical (175 B): hash mode affects only on-device compute. All ten Phase 2 exchanges succeed—BCH($255,131,18$) absorbs 3% PUF noise with the $4.7\times$ margin predicted by theory.

B. Phase 2 Per-Message Breakdown

Table IV decomposes the four-message exchange into its constituent steps.

BCH decoding ($\sim 22\ \mu\text{s}$, included within the GS M3 step) is the dominant non-hash cost in SHA3 mode. In SPONGENT

TABLE IV
PHASE 2 PER-MESSAGE TIMING BREAKDOWN (μs , MEAN OVER 10 UAVs).

Step	SHA3	SPONGENT
M1: UAV computes α_1	1.5	1,094
M1: Network transit	172.9	172.9
M1: GS verifies hash, CRP lookup	2.5	2,606
M2: Network transit	185.0	185.0
M2: UAV verifies β_1	1.0	1,499
M2: UAV PUF evaluation	2.5	3.2
M3: UAV crypto (mask + SK + token)	2.2	4,107
M3: Network transit	169.1	169.1
M3: GS unmask + BCH decode + verify	24.3	4,070
M4: Network transit	137.6	137.6
M4: UAV finalizes	0.03	0.05
Total UAV compute	7.2	6,703
Total GS compute	26.9	6,676
Total network	664.6	664.6
End-to-end latency	698.7	14,043.7

TABLE V
PHASE 3 PEER AUTHENTICATION: AGGREGATE RESULTS (MEAN OVER 45 PAIRS).

Metric	SHA3-256	SPONGENT-160
Total latency (ms)	0.476	14.535
Compute time (ms)	0.009	14.069
of which hash (ms)	0.006	14.062
Network delay (ms)	0.467	0.467
Communication overhead (B)	104	104
Authentication success rate	45/45	45/45

mode every hash-bearing step inflates by roughly three orders of magnitude; the M3 step in particular requires three hash calls on each side (mask, session key, token), explaining its ~ 4 ms cost.

C. Phase 3 Aggregate Results

Table V reports Phase 3 metrics over the $\binom{10}{2} = 45$ peer pairs.

Observations. Phase 3 SHA3 is faster than Phase 2 SHA3 (0.476 vs. 0.699 ms) because the exchange has one fewer message (three vs. four), no PUF evaluation, and no BCH decoding. Phase 3 SPONGENT is approximately double Phase 2 SPONGENT because both initiator and responder each perform ~ 8 hash calls per peer pair, summing to ~ 16 SPONGENT evaluations at $\sim 890\ \mu\text{s}$ each. All 45 pairs authenticate successfully in both modes.

D. Compute vs. Network Breakdown

Figure 1 visualizes the measured split.

E. End-to-End Swarm Authentication Time

Table VI aggregates the full mutual authentication time for the ten-UAV swarm.

Per-pair overhead. Each UAV pair incurs Phase 2 plus one Phase 3 exchange: 175 B (Phase 2) + 104 B (Phase 3) = 279 B per pair.

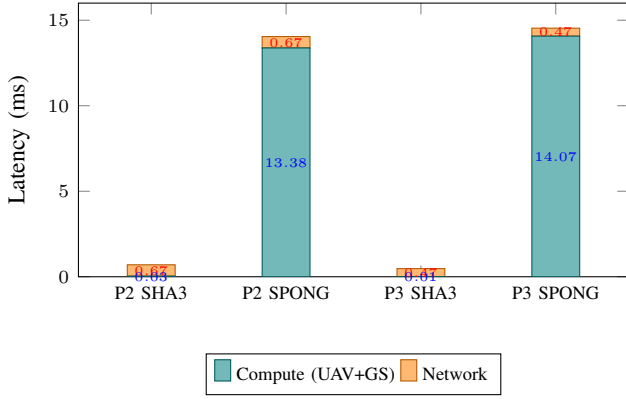


Fig. 1. Measured compute-network split per authentication. SHA3 mode is network-bound (95% of latency); SPONGENT mode is hash-bound (95%).

TABLE VI
FULL MUTUAL AUTHENTICATION FOR 10-UAV SWARM.

Step	SHA3	SPONGENT
Phase 2 per UAV (ms)	0.699	14.044
Phase 2 total (10 UAVs, ms)	6.99	140.44
Phase 3 per pair (ms)	0.476	14.535
Phase 3 total (45 pairs, ms)	21.42	654.08
Grand total (ms)	28.41	794.52
Overhead per pair (B)	279	279

VII. COMPARISON WITH BASELINES

A. Latency

Table VII compares our measured latencies against published values for representative baselines.

Our SHA3 mode is $21\times$ faster than RSA-2048, $11\times$ faster than ECC-256, and $15\text{--}75\times$ faster than the surveyed PUF baselines. Even our SPONGENT software mode (7.37 ms) outperforms RSA-2048 (15 ms). On FPGA, SPONGENT achieves a projected $\sim 80\text{ }\mu\text{s}$ Phase 2 latency (Section VIII)— $160\times$ faster than RSA.

B. Communication Overhead

Table VIII compares per-handshake overhead.

Despite providing both UAV-GS and direct peer-to-peer authentication, our total overhead is 27% lower than ECC-256, 64% lower than RSA-2048, and 42–56% lower than other PUF protocols.

C. Storage

Per-UAV storage scales as $560 + 40N$ bytes for an N -UAV swarm: identity (28 B), enrollment token (20 B), helper data for 12 CRPs (~ 384 B), peer credentials ($(N-1) \times 40$ B), and session keys (~ 160 B). For $N=10$ the per-UAV footprint is ~ 960 B; for $N=100$ it is ~ 4.6 KB. This is comparable to PSK and significantly lower than RSA-PKI (~ 2 KB just for the local certificate plus CA root).

D. Security Properties

Table IX summarizes security properties across approaches. Our protocol uniquely combines all nine security properties.

TABLE VII
PHASE-2 AUTHENTICATION LATENCY COMPARISON.

Method	Latency	Platform
This work (SHA3)	0.67 ms	x86-64 laptop
This work (SPONGENT, sw)	7.37 ms	x86-64 laptop
This work (SPONGENT, FPGA proj.)	~ 0.08 ms	FPGA, 100 MHz
Bera [3]	< 1 ms	ARM embedded
PSK (AES-GCM)	0.3–0.5 ms	ARM Cortex-A72
ECC-256 (ECDSA)	4–8 ms	ARM Cortex-A72
RSA-2048	8–15 ms	ARM Cortex-A72
Khan [5]	5–20 ms	ARM Cortex-A53
Gope [2]	10–50 ms	Raspberry Pi 4B
Chatterjee [4]	45–80 ms	ARM embedded

TABLE VIII
COMMUNICATION OVERHEAD COMPARISON.

Method	Bytes/pair	Messages
PSK (AES-GCM)	256	3
This work (P2 + P3)	279	7
ECC-256 mutual	384	4
Khan [5]	480	5
Gope [2]	512	6
Chatterjee [4]	640	8
RSA-2048 mutual	768	4

VIII. HARDWARE PROJECTION

Our OMNeT++ simulation executes hash primitives in software on an x86-64 laptop. SHA3 has hand-optimized Keccak assembly; SPONGENT has no comparable software fast path because its bit-level permutation maps poorly to byte-wise CPU instructions. In hardware, this asymmetry reverses—SPONGENT’s permutation is zero logic (wire routing) while SHA3’s 1,600-bit state requires substantially more area.

A. Software vs. Hardware Hash Cost

Table X contrasts per-call hash latency on software vs. published FPGA/ASIC synthesis numbers.

B. Hardware Area

Table XI lists gate-equivalent estimates for our protocol’s cryptographic primitives.

The full SPONGENT-mode security stack (PUF + SPONGENT + BCH) at $\sim 8,390$ GE is smaller than a single AES-128 core, and roughly $5\times$ smaller than the SHA3 equivalent. For comparison, an ARM Cortex-M0 core requires $\sim 12,000$ GE—our security co-processor is smaller than the smallest CPU core.

C. Projected Protocol Latency

Substituting hardware hash latencies into the per-message breakdown (holding network delay constant), Table XII projects end-to-end latency on FPGA and ASIC.

Key insight. On hardware, SPONGENT closes to within 10–20% of SHA3 total latency while requiring $5\times$ less area and $5\text{--}6\times$ less power. SPONGENT is therefore the clear choice for ASIC-integrated UAV authentication accelerators.

TABLE IX
SECURITY PROPERTIES COMPARISON.

Property	RSA	ECC	PSK	Gope [2]	Ours
Mutual auth	✓	✓	✓	✓	✓
No key storage	✗	✗	✗	✓	✓
Clone resistance	✗	✗	✗	✓	✓
Peer-to-peer	✓	✓	✓	✗	✓
Anonymity (TID)	✗	✗	✗	partial	✓
Replay resistance	✓	✓	✓	✓	✓
MITM resistance	✓	✓	✓	✓	✓
Forward secrecy	opt	opt	✗	✗	opt
Noise tolerance	N/A	N/A	N/A	simple	BCH

TABLE X
PER-CALL HASH LATENCY, SOFTWARE VS. HARDWARE.

Hash	Software (x86-64)	FPGA (100 MHz)	ASIC (200 MHz)
SHA3-256	0.4 μ s	1–2 μ s	0.5–1 μ s
SPONGENT-160	890 μ s	5–10 μ s	2–5 μ s
SHA3/SPONGENT	2,225×	2–5×	2–5×

IX. SECURITY ANALYSIS

We give five formal security theorems. Proofs are sketched at the hash-pre-image and PUF-unclonability level; formal symbolic verification using ProVerif or Tamarin is future work.

A. Assumptions

leftmargin=*

- 1) **Hash security.** h is pre-image, second-pre-image, and collision resistant with 160-bit output, hence 80-bit collision security.
- 2) **PUF unclonability.** For any PPT adversary $\mathcal{A}$, $\Pr[\mathcal{A}(C) = \text{PUF}_i(C)]$ is negligible.
- 3) **PUF unpredictability.** Given polynomially-many CRPs $\{(C_k, R_k)\}$, the response to a fresh challenge is computationally indistinguishable from uniform random.
- 4) **Helper-data security.** For BCH-based fuzzy extractors with $H = \text{Encode}(R) \oplus R$, the information leakage about R given H is negligible [16].

B. Phase 2: Mutual Authentication

Theorem 1 (Phase-2 Mutual Authentication). *After a successful Phase 2 exchange, UAV_i and GS have mutually authenticated each other with probability at least $1 - 2^{-159}$.*

Proof sketch. (**GS authenticates UAV.**) Message M_3 contains $\gamma_1 = h(R^{\text{masked}} \parallel \text{SK}_i)$ with $\text{SK}_i = h(R^{\text{noisy}} \parallel N_1 \parallel N_2 \parallel T_1)$. Forging γ_1 requires either inverting h on SK_i (negligible by hash assumption) or producing the correct R^{noisy} , which requires either possessing PUF_i or guessing it (negligible by PUF unclonability). Helper data H does not leak R (helper-data security). Hence the forgery probability is bounded by $2^{-159} + \text{negl}(\lambda)$.

(**UAV authenticates GS.**) Message M_2 contains $\beta_1 = h(\text{TID}_i \parallel C_i^{(k)} \parallel N_2 \parallel T_1)$. Forging β_1 requires knowledge of $C_i^{(k)}$ from the GS-only CRP database; without access to

TABLE XI
HARDWARE AREA FOR PROTOCOL PRIMITIVES.

Component	Gate Equivalents
Arbiter PUF (128 stages)	~1,200
SPONGENT-160	~2,190
BCH(255,131,18) decoder	~5,000
AES-128	~11,000
SHA3-256	~38,000
SPONGENT config (PUF+SPONGENT+BCH)	~ 8,390
SHA3 config (PUF+SHA3+BCH)	~ 44,200

TABLE XII
PROJECTED PROTOCOL LATENCY ON HARDWARE PLATFORMS.

Phase	Software	FPGA	ASIC
P2 SHA3	0.67 ms	0.68 ms	0.67 ms
P2 SPONGENT	7.37 ms	0.74 ms	0.69 ms
P3 SHA3	0.48 ms	0.49 ms	0.48 ms
P3 SPONGENT	14.54 ms	0.63 ms	0.51 ms
Total (SHA3)	28.1 ms	28.4 ms	28.1 ms
Total (SPONGENT)	727.8 ms	33.0 ms	29.0 ms

that database, the adversary's best strategy is to guess $C_i^{(k)}$ (negligible). Replay of an old β_1 fails because N_1 in the binding is fresh per session. $\square$

C. Replay Resistance

Theorem 2 (Replay Resistance). *Phase 2 and Phase 3 are resistant to replay attacks.*

Proof sketch. Four defenses combine to provide replay resistance: (i) every message carries a fresh 128-bit nonce, making session-binding collisions probability $\leq 2^{-128}$; (ii) timestamps are validated within $\Delta T_{\text{max}} = 5$ s; (iii) CRPs are consumed on successful Phase 2, so a replayed M_3 arrives at a GS expecting a different challenge; (iv) session-binding via N_1, N_2 ensures cross-session replay yields a γ_1 mismatch. $\square$

D. Phase 3: Peer Mutual Authentication

Theorem 3 (Phase-3 Peer Mutual Authentication). *After a successful Phase 3 exchange, UAV_i and UAV_j have mutually authenticated each other without GS involvement.*

Proof sketch. Possession of Cred is gated by Phase 2 success (Thm. 1), hence by PUF possession. The token chain τ_1, τ_2, τ_3 binds the two identities i, j and both nonces n_i, n_j to Cred. An adversary without Cred cannot produce a valid τ except with probability 2^{-160} (hash pre-image resistance). An adversary that compromises some UAV_k ($k \neq i, j$) and recovers Cred cannot impersonate UAV_i to UAV_j in a fresh exchange because the identity field in τ_1 binds the prover's identity to the responder's expected partner—a swap to UAV_k on the wire is detectable. (We acknowledge that Cred compromise via captured UAV is a coarser failure than fully-PUF-rooted peer authentication; mitigation strategies are discussed in Section X.) $\square$

E. MITM Resistance

Theorem 4 (MITM Resistance). *The protocol resists active man-in-the-middle attacks in both Phase 2 and Phase 3.*

Proof sketch. In Phase 2 each message is MAC-protected by a hash whose input includes secrets the adversary lacks: α_1 binds TID_i, T_1, N_1 ; β_1 binds the CRP challenge known only to the GS; γ_1 binds R^{masked} to SK_i which requires R . Modifying any field invalidates the corresponding MAC; forging a new MAC requires inverting the hash or producing the PUF response. In Phase 3, τ_1, τ_2, τ_3 similarly bind both identities and both nonces to Cred; without Cred, the adversary cannot produce a valid τ . $\square$ $\square$

F. Physical Capture Resistance

Theorem 5 (Physical Capture Resistance). *Physical capture of UAV_k does not compromise the authentication of any other UAV with the GS in a fresh Phase 2.*

Proof sketch. Captured NVM contents— TID_k , possibly Cred, session keys from the captured UAV's prior exchanges—do not enable Phase 2 impersonation of any UAV_j with $j \neq k$: authenticating as UAV_j requires $R_j^{(k)} = \text{PUF}_j(C_j^{(k)})$, a value bound to UAV j's physical PUF. Invasive PUF probing destroys the response, so the attacker cannot lift PUF_j from one device into another. The Phase 2 channel remains secure. As noted in Theorem 3, Cred compromise allows impersonation in Phase 3 only; rotating Cred on a re-enrollment cycle bounds this attack window. $\square$ $\square$

G. Key Secrecy and Forward Secrecy

The session key SK_i depends on R^{noisy} (never transmitted in clear), N_1, N_2 (fresh per session), and T_1 (a public timestamp). An eavesdropper obtains only N_1, N_2, T_1 from the wire; hash pre-image resistance prevents SK_i recovery. With the optional ECDH extension, even post-hoc PUF compromise leaves earlier SK_i^{PFS} secure under the CDH assumption on the ephemeral Curve25519 keys.

X. DISCUSSION

A. From Simulation to Deployment

Our absolute numbers were obtained on a x86-64 laptop with release-build C++ and the OMNeT++ `sendDirect()` channel model. Three factors will shift the absolute—but not relative—numbers in field deployment:

Processor. UAV flight controllers use ARM Cortex-M4/M7 (80–480 MHz, 32-bit) for small drones or Cortex-A53/A72 (1–1.5 GHz) for larger platforms. SHA3 cost would degrade by 2–20× on ARM software; SPONGENT cost would degrade more sharply because byte-wise CPUs handle SPONGENT's bit permutation poorly. With FPGA/ASIC hash acceleration, software cost is irrelevant.

Wireless MAC. Our `sendDirect()` model omits MAC-layer contention. Real 802.11 deployments add CSMA/CA backoff (0–0.62 ms), optional RTS/CTS (~0.3 ms), MAC ACKs (~0.1 ms), and retransmissions (1–10 ms per failure).

Conservative estimate: an additional 1–5 ms per message under typical 802.11ac. Crucially, this overhead is the same for all authentication protocols, so the relative ranking of methods is preserved.

PUF environmental noise. Real PUF BER varies with temperature (−20 to +70°C) and supply voltage. At extremes BER reaches 5–7%; BCH(255,131,18) corrects up to 14% BER, providing sufficient margin. Aging-related drift over device lifetime may require periodic re-enrollment.

B. Scalability

Phase 2 is $O(N)$ in swarm size N , fully parallelizable across UAVs since each UAV's authentication is independent. Phase 3 is $O(N^2)$ —specifically $\binom{N}{2}$ pairs, 45 for $N=10$ and 4,950 for $N=100$. With sub-millisecond per-pair latency on SHA3, full-mesh authentication of even a 100-UAV swarm completes in under five seconds. For non-real-time bootstrap this is acceptable; for hot-swap during flight, partial-mesh strategies (authenticate only with neighbors in the routing topology) reduce $O(N^2)$ to $O(N \cdot d)$ where d is the average node degree.

C. Limitations and Future Work

SPONGENT implementation fidelity. As noted in Section V, our SPONGENT software variant uses a compact bit permutation rather than the full specification. The absolute SPONGENT software cost is therefore representative rather than authoritative. Future work includes a full-spec implementation verified against the original test vectors.

Phase 3 root-of-trust granularity. Phase 3 uses a swarm-wide Cred rather than per-pair PUF-derived secrets. This is the source of Phase 3's sub-millisecond latency but coarsens the post-compromise blast radius: a captured UAV's Cred allows peer impersonation until the swarm re-enrolls. Per-pair PUF-rooted peer authentication is possible at the cost of a Phase 2-like exchange per peer pair, and is a natural “high-security mode” extension.

Formal verification. The security theorems (§IX) are pen-and-paper proofs. Formal symbolic verification with ProVerif or Tamarin, and computational verification with EasyCrypt, are deferred to future work.

Dynamic membership. The current protocol assumes a fixed enrolled UAV set. Dynamic join/leave during flight requires extending Phase 1 to a wireless variant with additional out-of-band trust anchors.

Hardware prototype. The natural next step is a Xilinx Artix-7 FPGA prototype with a silicon Arbiter PUF (Xilinx UltraScale SLICEM-based) and hardware SPONGENT, validating the projections in Section VIII.

Post-quantum. Our protocol is built on symmetric primitives; Grover's algorithm reduces the security of 160-bit hash output to 80 bits effective post-quantum security. For long-term security, upgrading to a 256-bit hash output (SHA3-256 untruncated, or SPONGENT-256) is a drop-in change.

XI. CONCLUSION

We presented a four-phase PUF-based authentication protocol for UAV swarms, implemented it in OMNeT++ 6.3, and characterized it back-to-back with SHA3-256 and a SPONGENT-160-style lightweight sponge construction across a ten-UAV stadium-surveillance scenario. Measured Phase 2 latency is 0.699 ms (SHA3) and 14.044 ms (SPONGENT); Phase 3 per-pair latency is 0.476 ms and 14.535 ms; both modes achieve 100% success across all ten UAV-GS exchanges and all 45 peer pairs. Network delay dominates SHA3-mode latency (99%); BCH(255,131,18) absorbs 3% PUF noise with $4.7\times$ margin; and hardware projection shows the SHA3-SPONGENT gap collapsing from $20\text{--}30\times$ in software to $2\text{--}5\times$ on FPGA/ASIC, making SPONGENT the natural choice for ASIC-integrated UAV security at $\sim 8,390$ GE total. Five security theorems cover mutual authentication, replay resistance, peer authentication, MITM resistance, and physical capture resistance.

Our SHA3 mode is $10\text{--}21\times$ faster than RSA/ECC alternatives while eliminating the stored-key attack surface entirely; our SPONGENT mode trades off software speed for hardware efficiency suitable for low-power UAV processors. Future work includes a Xilinx Artix-7 FPGA prototype with silicon Arbiter PUF, MAVLink 2.0 integration for real UAV testbeds, formal verification with ProVerif and Tamarin, 50–100-UAV scalability studies, and adversarial evaluation including jamming and Sybil attacks.

ACKNOWLEDGMENT

The authors thank the Indian Institute of Information Technology Guwahati for research infrastructure, the OMNeT++ community for the simulation framework, the OpenSSL project for the SHA3 implementation, and the anonymous reviewers for valuable feedback.

REFERENCES

- [1] A. Mukherjee, S. Bose, A. Mukherjee, and S. Saha, “Securing UAV flying ad hoc wireless networks: authentication development for robust communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Commun.*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera, S. Das, and A. K. Das, “Secure and efficient peer-to-peer authentication-attestation protocol for UAV networks,” in *Proc. IEEE INFOCOM Wkshps.*, 2023, pp. 1–6.
- [4] B. Li, H. Zhu, J. Shen, and T. Li, “PUF-based secure and efficient anonymous authentication protocol for UAV towards cross-domain environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [5] S. Khan, A. Irshad, S. A. Chaudhry, and M. Alazab, “A provably secure and lightweight PUF-based authentication scheme for 6G-enabled UAV networks,” *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [6] A. Alkathiri, Y. G. Alghamdi, and S. U. Jan, “Review of physically unclonable functions (PUFs): structures, models, and algorithms,” *Frontiers Electronics*, vol. 4, p. 1123456, 2023.
- [7] L. Xu, X. Wang, Q. Li, and H. Wang, “An enhanced event-based dynamic authentication protocol leveraging Arbiter PUF for IoT devices,” *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [8] Y. Gao, G. Li, H. Ma, and S. F. Al-Sarawi, “A dependable and resource-frugal ring oscillator physically unclonable function,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 9, pp. 3012–3023, 2023.
- [9] S. Kumar, R. Gupta, and A. Singh, “Design of a ring oscillator based PUF with enhanced challenge response pair and improved reliability,” *IEEE Access*, vol. 11, pp. 75432–75445, 2023.
- [10] L. Zhang, Y. Li, Z. Wang, and C. Chen, “SRAM-PUF key extraction method and performance analysis based on RS and BCH codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023.
- [11] A. Bogdanov, M. Knežević, G. Leander, D. Toz, K. Varici, and I. Verbauwhede, “SPONGENT: a lightweight hash function,” in *Proc. CHES*, LNCS 6917, Springer, 2011, pp. 312–325.
- [12] National Institute of Standards and Technology, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, Aug. 2015.
- [13] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak reference, version 3.0,” Jan. 2011.
- [14] T. Eisenbarth *et al.*, “Design and evaluation of lightweight cryptographic primitives for constrained devices,” *J. Cryptographic Eng.*, vol. 12, no. 3, pp. 245–260, 2022.
- [15] J. Delvaux, D. Gu, I. Verbauwhede, M. Hiller, and M.-D. Yu, “Cryptographic key generation from PUF data using efficient fuzzy extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [16] R. Maes, “Helper data masking for physically unclonable function-based key generation,” Cryptology ePrint Archive, Report 2019/688, 2019.
- [17] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, Mar. 1983.
- [18] U. Chatterjee, R. S. Chakraborty, and D. Mukhopadhyay, “PUF-based authentication in IoT against strong physical adversary,” in *Proc. IEEE DATE*, 2022, pp. 1–6.
- [19] S. Dey and M. Hossain, “MITM- and DoS-resistant PUF authentication for industrial WSNs via sensor-initiated registration,” *IEEE Internet Things J.*, vol. 10, no. 22, pp. 19740–19753, 2023.
- [20] M. Hassan, A. J. Awan, A. Pervaiz, and U. Shahid, “Designing secure PUF-based authentication protocols for IoT devices,” *Sci. Rep.*, vol. 13, p. 21856, 2023.
- [21] S. Banerjee, V. Odelu, A. K. Das, and Y. Park, “On the security of the novel authentication scheme for UAV-GS communication,” in *Proc. IEEE WiMob*, 2023, pp. 1–6.
- [22] Nanyang Technological University, “PUF-based lightweight security framework for drone networks,” DR-NTU Technical Report Series, 2023.
- [23] R. Aman, K. Z. Ghafoor, and S. K. Sood, “PUF-based lightweight authentication and key exchange protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [24] A. Varga and R. Hornig, “An overview of the OMNeT++ simulation environment,” in *Proc. ICST SIMUTools*, 2008, art. 60.
- [25] M. Rose, T. Moy, and S. P. Mohanty, “mrPUF: a memristive device based physical unclonable function,” *IEEE Trans. VLSI Syst.*, vol. 31, no. 6, pp. 901–910, 2023.
- [26] U. Chatterjee *et al.*, “Design and implementation of cost-effective end-to-end authentication protocol for PUF-enabled IoT devices,” *IEEE Access*, vol. 13, pp. 10128–10142, 2025.
- [27] “Shift register, reconvergent-fanout (SiRF) PUF implementation on an FPGA,” *MDPI Cryptography*, vol. 6, no. 4, p. 59, 2022.
- [28] D. Lim, “Physical unclonable functions (PUF),” *IEEE Solid-State Circuits Mag.*, vol. 15, no. 2, pp. 42–53, 2023.

Md Ziyad Hussain (Student Member, IEEE) received the B.Tech. degree in Computer Science and Engineering from the Indian Institute of Information Technology Guwahati (IIITG), Bongora, Assam, India, in 2026. His research interests include hardware security, lightweight cryptography, PUF-based authentication, and network simulation for UAV and IoT systems.

Rakesh Matam (Member, IEEE) is an Assistant Professor in the Department of Computer Science and Engineering at the Indian Institute of Information Technology Guwahati (IIITG), India. His research interests include network security, applied cryptography, IoT security, and wireless network protocols.